

1. Signale

1. Wert und Zeit

Aufgabe: Entscheiden Sie für die folgenden Signale, ob sie wertkontinuierlich oder wertdiskret, sowie zeitkontinuierlich oder wertdiskret sind. Begründen Sie kurz.

- Der Schall, den unsere Ohren hören.
- Die Anzahl an Menschen in einem Museum.
- Die Mittagstemperatur in Potsdam.
- Die Anzahl verkaufter Produkte pro Tag.

Lösung

Signal	Wertachse	Zeitachse	Begründung
Der Schall, den unsere Ohren hören.	wertkontinuierlich	zeitkontinuierlich	Die physikalische Größe (Druck) kann unendlich viele Werte zu jedem beliebigen Zeitpunkt annehmen.
Die Anzahl an Menschen in einem Museum.	wertdiskret	zeitkontinuierlich	Die Anzahl ist eine abzählbare Menge (ganze Zahlen). Änderungen erfolgen zu beliebigen Zeitpunkten (Eintritt/Austritt).
Die Mittagstemperatur in Potsdam. Annahme: Eine Messung pro Tag um genau 12:00.	wertkontinuierlich	zeitdiskret	Die Temperatur selbst ist kontinuierlich, wird aber nur zu einem einzelnen, festgelegten Zeitpunkt gemessen (Mittag).
Die Anzahl verkaufter Produkte pro Tag.	wertdiskret	zeitdiskret	Die Menge der verkauften Produkte ist eine ganze Zahl (wertdiskret). Die Feststellung erfolgt diskret pro Tag.

2. Absoluter Fehler

Antwort: Gehen wir von einer Sample-and-Hold Rekonstruktion aus, beträgt der absolute Fehler 1.

Lösung

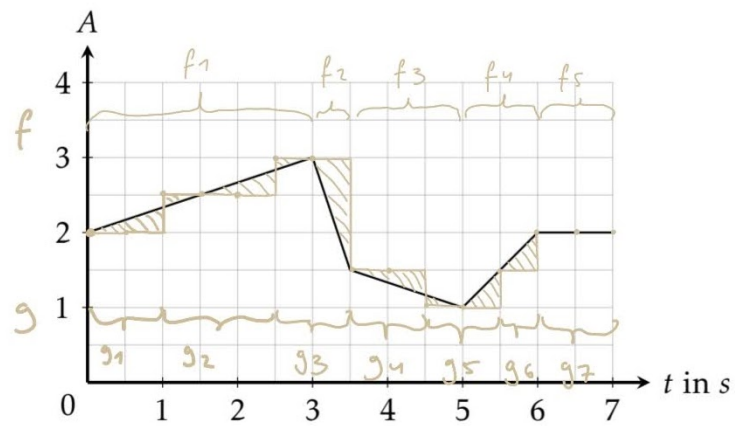


Abbildung 1: Verlauf des Signals A über der Zeit t

A1

2) Absoluter Fehler

D (siehe Abb. 1 u. 1.2)

1) Wert u. Zeithont Sig

2) Wert u. Zeitdiskretes Sig

Q

1) Absoluter Fehler

A

1) 1

L

1) Wir suchen:

$$\left| \int_0^3 f_1 - \int_0^3 g \right| +$$

$$\left| \int_3^{3,5} f_2 - \int_3^{3,5} g \right| +$$

$$\left| \int_{3,5}^5 f_3 - \int_{3,5}^5 g \right| +$$

$$\left| \int_5^6 f_4 - \int_5^6 g \right| +$$

$$\left| \int_6^7 f_5 - \int_6^7 g \right| = \delta$$

2) Wir bestimmen f_{1-5} u. ihre Stammfunktionen.

$$f_1 = \frac{1}{3}x + 2 \quad F_1 = \frac{1}{6}x^2 + 2x$$

$$f_2 = -\frac{\frac{3}{2}}{\frac{1}{2}} + 6$$

$$3 = -3 \cdot 3 + 6 \Leftrightarrow 6 = 12$$

$$f_2 = -3x + 12 \quad F_2 = -\frac{3}{2}x^2 + 12x$$

$$f_3 = -\frac{\frac{1}{2}}{\frac{3}{2}} + b$$

$$5 = -\frac{1}{3} + b \Leftrightarrow b = 5\frac{1}{3}$$

$$f_3 = -\frac{1}{3} + 5\frac{1}{3} \quad F_3 = -\frac{1}{6}x^2 + 5\frac{1}{3}x$$

$$f_4 = 1 \cdot x + b$$

$$1 = 5 + b \Leftrightarrow b = -4$$

$$f_4 = x - 4 \quad F_4 = \frac{1}{2}x^2 - 4x$$

$$f_5 = 2 \quad F_5 = 2x$$

3a) Wir integrieren f

$$\int_0^3 f_1 = \left[\frac{1}{6} x^2 + 2x \right]_0^3 = \left(\frac{1}{6} \cdot 9 + 6 \right) - 0 = \frac{15}{2}$$

$$\begin{aligned} \int_3^{3,5} f_2 &= \left[-\frac{3}{2} x^2 + 12x \right]_3^{3,5} = \left(-\frac{3}{2} \cdot \left(\frac{7}{2} \right)^2 + 12 \cdot \frac{7}{2} \right) - \left(-\frac{3}{2} \cdot 9 + 36 \right) = -\frac{3 \cdot 49}{8} + 42 + \frac{27}{2} - 36 \\ &= -\frac{147}{8} + 6 + \frac{27}{2} = \frac{48}{8} + \frac{108}{8} - \frac{147}{8} = \frac{9}{8} \end{aligned}$$

$$\int_{3,5}^5 f_3 = \frac{15}{8}$$

$$\int_5^6 f_4 = \frac{3}{2}$$

$$\int_6^7 f_5 = 2$$

3b) Und g

$$\int_0^3 g = 2 + \frac{3}{2} \cdot \frac{5}{2} + \frac{3}{2} = \frac{7}{2} + \frac{15}{4} = \frac{29}{4}$$

$$\int_3^{3,5} g = \frac{3}{2}$$

$$\int_{3,5}^5 g = 2$$

$$\int_5^6 g = \frac{9}{4}$$

$$\int_6^7 g = 2$$

4) Und summieren die Beträge d. Differenzen

$$\delta_1: \left| \frac{15}{2} - \frac{29}{4} \right| = \frac{1}{4} = \frac{2}{8}$$

$$\delta_2: \left| \frac{9}{8} - \frac{3}{2} \right| = \frac{3}{8}$$

$$\delta_3: \left| \frac{15}{8} - 2 \right| = \frac{1}{8}$$

$$\delta_4: \left| \frac{3}{2} - \frac{5}{4} \right| = \frac{1}{4} = \frac{2}{8}$$

$$\delta_5: 0$$

$$\delta = \frac{2}{8} + \frac{3}{8} + \frac{1}{8} + \frac{2}{8} = 1$$

3. Analyse des Ergebnisses

Wir können bei Quantifizierungen einen Fehler erwarten. Dieser ist allerdings bei einem linearen Signal vorhersagbar und hängt nicht von der Abtastrate ab. Auch könnten wir ein lineares Signal aus nur zwei Messungen perfekt rekonstruieren (wenn wir linear interpolieren). Dagegen hängt der Fehler bei einem nicht-linearen Signal zusätzlich zum Quantifizierungsschema maßgeblich von der Abtastrate ab (s. Abbildung). Die Rekonstruktion ist komplizierter und kann nur ordentlich gelingen, wenn der höchste Frequenzanteil kleiner ist als die Hälfte der Abtastrate.

4. Größtmöglicher Fehler

Der größtmögliche Fehler ist halb so groß wie die Quantisierungsstufe. Bei größerer Abweichung wird wiederum auf die nächstliegende QStufe gerundet.

2. Zahldarstellungen

1. Stellen Sie die Zahlen 36_{10} , -36_{10} und -128_{10} dar. Inwiefern ändert sich die Zahldarstellung, wenn man mehr als 8 Bits verwenden würde?

Dezimal	8-Bit	10-Bit
36_{10}	00100100	0000100100
-36_{10}	11011100	1111011100
-128_{10}	10000000	1110000000

Bei mehr Bits werden negative Zahlen von links mit Einsen aufgefüllt.

2. Bestimmen Sie die kleinste und größte darstellbare Zahl.

$$MIN_8 = -128_{10} = 10000000$$

$$MAX_8 = 127_{10} = 01111111$$

3. Erklären Sie, warum das Negieren der kleinsten Zahl nicht darstellbar ist.

Im Zweierkomplement-System ist Negation definiert als $NOT(a) + 1$. Wenn wir das mit der kleinsten Zahl -128 machen, kommen wir wieder bei -128 raus!

$$NEG(10000000) = NOT(10000000) + 1 = 01111111 + 1 = 10000000$$

4. Zeigen Sie, dass für alle Zahlen a im Zweierkomplement gilt: $-a = NOT(a) + 1$.

Wenn wir jedes Bit einer Zahl a negieren, erhalten wir eine Zahl $NOT(a)$, die zu a addiert eine Zahl ergibt, die nur aus Einsen besteht: $a + NOT(a) = 11 \dots 1$.

Wir wissen, dass im Zweierkomplement eine Zahl nur aus Einsen immer den Wert -1 hat. Also sehen wir dass

$$a + NOT(a) = -1 \iff -a = NOT(a) + 1$$

Folgend gehen wir von IEEE-754 single precision aus.

1. Stellen Sie die IEEE-754 Zahl $C27FC000$ als Dezimalzahl dar.

Zunächst stellen wir die Zahl in binärer Form dar. Da das Hexadezimalsystem $16 = 2^4$ als Base hat, können wir jede Hex-Ziffer als eine Folge von 4 Bits darstellen. Wenn wir diese aneinanderreihen, erhalten wir die ganze Zahl. Mit $C = 12 = 1100$, $2 = 0010$, $7 = 0111$ und $F = 15 = 1111$ ergibt das die folgende 32-bit Zahl:

$$11000010011111111100000000000000$$

Aufgeteilt in Vorzeichen-Bit(s), Mantisse(m) und Exponent(e):

- $s = 1$
- $e = 10000100 - 127 = (128 + 4) - 127 = 5$
- $m = 1 + 0.111111111100000000000000_2 = 1 + \frac{511}{512} = 1.99804687$

Damit berechnen wir unsere Zahl laut Formel $(-1)^s \cdot m \cdot 2^e$:

$$(-1) \cdot 1.99804687 \cdot 2^5 = -63.9375$$

2. Stellen Sie die Fließkommazahl 42.875 als Binärzahl dar.

Wir brechen zunächst auf in ganze Zahl und Bruch:

$$42 = 32 + 8 + 2 = 101010_2$$

$$0.875 = \frac{7}{8} = 4/8 + 2/8 + 1/8 = 1/2 + 1/4 + 1/8 = 0.111_2$$

Zusammen ergibt das in normalisierter Form:

$$42.875 = 101010.111_2 = 1.01010111_2 \cdot 2^5$$

Wir erinnern uns, dass single precision Zahlen repräsentiert werden als

$$(-1)^s \cdot (1 + \text{Mantisse}) \cdot 2^{\text{Exponent} - 127}$$

und kommen in unserem Fall auf

$$(-1)^0 \cdot (1 + 0.010101110000000000000000) \cdot 2^{132-127}$$

In single precision repräsentieren wir die Zahl 42.875 also durch

$$\underbrace{0}_{\text{Vorzeichen}} \quad \underbrace{10000100}_{\text{Exponent}} \quad \underbrace{010101110000000000000000}_{\text{Mantisse}}$$

1 3. UTF-8

1. Wie viele Zeichen sind in UTF-8 darstellbar? Wieso ist das in der Praxis weniger als theoretisch möglich?

UTF-8 erlaubt Codepoint-Sequenzen von 1 bis 4 Bytes. Der größte theoretisch darstellbare Wert wäre alle 4 Bytes mit Einsen aufzufüllen:

11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Figure 1: **K**ontrollbits, **W**ertbits

21 Wertbits ergeben $2^{21} - 1$, also ca. 2,1 Millionen.

Tatsächlich sind laut Unicode-Standard etwa 1,1 Millionen Zeichen darstellbar.

Woher die Diskrepanz?

Gründe für die Reduzierung: Die kurze Antwort lautet: weil es die Unicode Spezifikation so definiert¹:

Each encoding form maps the Unicode code points U+0000..U+D7FF and U+E000..U+10FFFF to unique code unit sequences.

(*The Unicode Standard, Version 16.0, Chapter 3*)

Primärquellen für diese Entscheidung sind nur schwer aufzufinden. Sekundärquellen scheinen sich einig darüber, dass die Anzahl an darstellbaren Zeichen in UTF-8 reduziert wurde, um das Encoding mit UTF-16 kompatibel zu machen. UTF-16 hat einen beträchtlich kleineren Spielraum. Wären gewisse Zeichen nur durch UTF-8 aber nicht durch UTF-16 darstellbar, würde die Komplexität des Systems in die Höhe schießen und das Ziel einer einheitlichen Darstellung von Zeichen wäre misslungen.

Außerdem repräsentiert die Obergrenze von 10FFFF eine Sicherheitsmaßnahme. Die UTF-8 Spezifikation (RFC 3629) begründet²:

Another security issue occurs when encoding to UTF-8: the ISO/IEC 10646 description of UTF-8 allows encoding character numbers up to U+7FFFFFFF, yielding sequences of up to 6 bytes. There is therefore a risk of buffer overflow if the range of character numbers is not explicitly limited to U+10FFFF or if buffer sizing doesn't take into account the possibility of 5- and 6-byte sequences

Weitere Reduktion ergibt sich aus der Auslassung von Codepoint-Sequenzen U+D800..U+DFFF. Dies hat wiederum damit zu tun, dass dieses Intervall für sog. Surrogate in UTF-16 reserviert sind. (Surrogate sind in UTF-16 zwei 16-Bit-Einheiten, die es UTF-16 ermöglichen, Codepoints jenseits von FFFF zu

¹<https://www.unicode.org/versions/Unicode16.0.0/core-spec/chapter-3/#G740>

²<https://datatracker.ietf.org/doc/html/rfc3629#section-1>

kodieren.) Insgesamt werden 2048 gesperrt. Damit dominiert die Obergrenze bei Weitem.

Weitere Quellen:

- Ein Brief der Unicode Gruppe an das ISO Konsortium, in dem sie bitten, die Range der Code Positions auf 10FFFF zu reduzieren³

2. Bytestream

Aufgabe:

Gegeben sei folgender Ausschnitt eines Bytestreams: 41 C3 84 E4 B8 AD C3 A9 CE A9. Bestimmen Sie für jedes Byte, ob es ein Startbyte oder ein Folgebyte ist. Folgern Sie, wie viele Zeichen die Nachricht enthält

Lösung:

Wir stellen die Bytes in Binärform dar.

Startbytes beginnen mit 0, 10, 110, 1110.

41 = 4 * 16 + 1	= 0100 0001	Startbyte
C3 = 12 * 16 + 3	= 1100 0011	Startbyte
84 = 8 * 16 + 4	= 1000 0100	Folgebyte
E4 = 14 * 16 + 4	= 1110 0100	Startbyte
B8 = 11 * 16 + 8	= 1011 1000	Folgebyte
AD = 10 * 16 + 13	= 1010 1101	Folgebyte
C3 = 12 * 16 + 3	= 1100 0011	Startbyte
A9 = 10 * 16 + 9	= 1010 1001	Folgebyte
CE = 12 * 16 + 14	= 1100 1110	Startbyte
A9 = 10 * 16 + 9	= 1010 1001	Folgebyte

5 Startbytes entspricht 5 Zeichen:

41
C384
E4B8AD
C3A9
CEA9

Das ergibt die Nachricht ΑΆ中éΩ.

³<https://www.unicode.org/L1/L2000/00079-n2175.html>

3. Selbstsynchronisation

Aufgabe:

UTF-8 ist ein selbstsynchronisierendes Kodierungsformat, das bedeutet es ist möglich, beim Lesen eines Bytestroms an einer beliebigen Stelle einzusteigen und die Grenzen der Zeichen zu erkennen. Erläutern Sie, wie dies möglich ist.

Lösung:

Wie wir in der vorigen Aufgabe gesehen haben, kann ein Programm an jedem Byte ablesen, ob es sich um ein Startbyte oder Folgebyte handelt. Wenn das Programm in der Mitte eines Zeichens landet, kann es einfach den Stream weiterlesen, bis es ein Byte findet, das mit einer Startsequenz beginnt und von da an parsen.

4. Bytegrenzen

Aufgabe:

Erklären Sie, warum dasselbe nicht immer möglich ist, wenn man ohne Kenntnis der Bytegrenzen bei einem zufälligen Bit beginnt zu lesen.

Lösung:

Das UTF-8 Schema basiert darauf, dass die Kontrollbits immer am Anfang eines Bytes stehen. Wenn wir nicht wissen, wo ein Byte anfängt, bricht das System zusammen: die Applikation kann ein Bit nicht mehr eindeutig als Kontrollbit identifizieren. Beispiel: Die Byte-Sequenz 11000011 10000100 (Ä) wird zu 10000111 0000100x... wenn man bei Bit 2 startet - die Kontrollmuster des Originals sind nicht mehr erkennbar.

4. Erzeugendensysteme

1. NAND und NOR

NAND aus NOR

Frage: Können wir $\neg$ durch NOR darstellen?

a	$\neg a$	$\overline{a \vee a}$
1	0	0
0	1	1

$$\rightarrow \neg a \equiv \overline{a \vee a}$$

Wir vergleichen mögliche Verkettungen:

a	b	$\overline{a \wedge b}$	$\overline{\overline{a \vee a} \vee \overline{b \vee b}}$
0	0	1	1
0	1	1	0
1	0	1	0
1	1	0	0

Aha! Wir sehen sofort dass

$$\overline{a \wedge b} = \neg (\overline{\overline{a \vee a}} \vee \overline{\overline{b \vee b}})$$

Und da $\neg a = \overline{\overline{a}}$ können wir umformen:

$$\overline{a \wedge b} = \overline{\overline{\overline{a \vee a}} \vee \overline{\overline{\overline{b \vee b}}}}$$

76) NOR aus NAND

Hierangeherweise: Algebra statt WWT.

Wir wissen dass $\overline{a \vee b} = \neg a \wedge \neg b$

und $\overline{a \wedge b} = \neg(a \wedge b) = \neg a \vee \neg b$

Untersuchen wir NAND und Negation:

a	$\neg a$	$\overline{a \wedge a}$
0	1	1
1	0	0

Aha! Auch hier wieder: $\neg a = \overline{a \wedge a}$

Dann können wir umformen:

$$\begin{aligned}
\overline{a \vee b} &= \neg a \wedge \neg b \\
&= \neg(\neg a \wedge \neg b) \\
&= \neg(\overline{\overline{a \wedge a}} \wedge \overline{\overline{b \wedge b}}) \\
&= \overline{\overline{(\overline{a \wedge a}) \wedge (\overline{b \wedge b})}}
\end{aligned}$$

Es fällt auf, dass

- beide Formeln dieselbe Struktur haben.

Ergo könnten wir f. beide dasselbe Gatter bauen und müssten nur die Gates austauschen um auf das andere zu kommen.

- Dualität ☺

2. Erzeugendensysteme

Aufgabe: Sind die folgenden Operatormengen Erzeugendensysteme? Falls ja, stelle NAND und NOR mit ihnen dar.

Lösung:

Wir kennen als Erzeugendensysteme NOR, NAND und AND, OR, NOT (aus Mathe 1). Beobachtung: Alle bekannten Erzeugendensysteme enthalten Negation.

Ansatz: Die jeweilige Operatormenge ist ein Erzeugendensystem, wenn wir eins der bekannten Systeme mit ihr darstellen können. Umgekehrt können wir eine Operatormenge ausschließen, wenn sich mit ihr nicht negieren lässt.

AND, OR

Bei Eingabe $(0, 0)$, ergibt jede Kombination von AND und OR ebenfalls 0. Da $\text{NOT}(0) = 1$, können wir nicht negieren. Die Menge ist kein Erzeugendensystem.

AND, NOT

NAND besteht per Definition aus AND und NOT: $a \uparrow b = \neg(a \wedge b)$. Da NAND ein Erzeugendensystem ist, ist diese Menge auch eins. Wir stellen NOR dar, indem wir bemerken, dass NOR nur dann 1 ist, wenn beide Eingaben 0 sind: $a \downarrow b = \neg a \wedge \neg b$

XOR, OR

Wie bei AND, OR, ist bei einer Eingabe von $(0, 0)$ die Ausgabe immer 0. Wir können wieder nicht negieren und die Operatormenge ist kein Erzeugendensystem.

XOR, OR, TRUE

Wir bemerken, dass $\neg a = a \oplus \top$, und kommen damit zu NOR:

$$a \downarrow b = \neg(a \vee b) = (a \vee b) \oplus \top$$

Um NAND zu erhalten, formen wir um:

$$\begin{aligned} a \uparrow b &= \neg(a \wedge b) \\ &= \neg a \vee \neg b && \text{(De Morgan)} \\ &= (a \oplus \top) \vee (b \oplus \top) && \text{(siehe oben)} \end{aligned}$$

3. DNF

X			$f(X)$				$\min$								DNF			
a	b	c	$((a \leftrightarrow b) \rightarrow \neg c)$	$\neg a$	$\neg b$	$(a \vee \neg b)$	0	1	2	3	4	5	6	7	$a \vee (\neg b \wedge \neg c)$			
0	0	0	1	1	1	1	1	1							0	1	1	1
0	0	1	1	1	0	1		1							0	0	1	0
0	1	0	0	1	1	0			1						0	0	0	1
0	1	1	0	1	0	1				1					0	0	0	0
1	0	0	0	0	1	1					1				1	1	1	1
1	0	1	0	0	0	1						1			1	1	1	0
1	1	0	1	0	0	1							1		1	0	0	1
1	1	1	1	0	0	1								1	1	0	0	0

4.4

Wir sehen dass $f(X) = m_0 \vee m_4 \vee m_5 \vee m_6 \vee m_7$, also

$$f(X) = (\bar{a} \wedge \bar{b} \wedge \bar{c}) \vee (a \wedge \bar{b} \wedge \bar{c}) \vee (a \wedge \bar{b} \wedge c) \vee (a \wedge b \wedge \bar{c}) \vee (a \wedge b \wedge c)$$

Wir vereinfachen durch Absorption:

$$\begin{aligned} f(X) &= (\bar{b} \wedge \bar{c}) \vee (a \wedge \bar{b} \wedge c) \vee (a \wedge b) \\ &= \bar{b} \wedge (\bar{c} \vee (a \wedge c)) \vee (a \wedge b) \\ &= \bar{b} \wedge (\bar{c} \vee a) \vee (a \wedge b) \\ &= (\bar{b} \wedge \bar{c}) \vee (\bar{b} \wedge a) \vee (a \wedge b) \\ &= (\bar{b} \wedge \bar{c}) \vee (a \wedge (b \vee \bar{b})) \\ &= a \vee (\bar{b} \wedge \bar{c}) \end{aligned}$$

Absorptionsregeln

$$a \vee (\bar{a} \wedge b) = a \vee b$$

$$(a \vee b) \wedge (a \vee \bar{b}) = a$$

Alternative zur Vereinfachung: Karnaugh-map

	$\neg b$	$\neg c$	$\neg b$	c	b	c	b	$\neg c$
$\neg a$	x							
a	x		x		x		x	

$$f(x) = a \vee (\neg b \wedge \neg c)$$

4. DNF Test

Die vereinfachte DNF ist logisch äquivalent zur ursprünglichen Aussage, siehe grün-markierter Bereich in der ersten Tabelle in vorangegangener PDF.

Aufgabe 5. Prioritäts-Multiplexer

⑤

PMUX

input				Data		
	S_3	S_2	S_1	S_0	d	$X_1 \quad X_0$
1	0	0	0	0	0	* *
2	0	0	0	1	d_0	0 0
3	0	0	1	0	d_1	0 1
4	0	0	1	1	d_1	0 1
5	0	1	0	0	d_2	1 0
6	0	1	0	1	d_2	1 0
7	0	1	1	0	d_2	1 0
8	0	1	1	1	d_2	1 0
9	1	0	0	0	d_3	1 1
10	1	0	0	1	d_3	1 1
11	1	0	1	0	d_3	1 1
12	1	0	1	1	d_3	1 1
13	1	1	0	0	d_3	1 1
14	1	1	0	1	d_3	1 1
15	1	1	1	0	d_3	1 1
16	1	1	1	1	d_3	1 1

Dann:

Zwei K-Map für X_1 und X_0 .

Für 4 Variablen (S_3, S_2, S_1, S_0) - 4x4 map.
(2^4 Zellen)

$\downarrow S_3 S_2 \leftrightarrow S_1 S_0$

Gray-Code: $\rightarrow 00 \ 01 \ 11 \ 10$

X_1	X_0	D
0	0	d_0
0	1	d_1
1	0	d_2
1	1	d_3

X_1 K-map:

		$S_1 S_0$			
		00	01	11	10
$S_3 S_2$	00	*	0	0	0
	01	1	1	1	1
	11	1	1	1	1
	10	1	1	1	1

X_0 K-map:

		$S_1 S_0$			
		00	01	11	10
$S_3 S_2$	00	*	0	1	1
	01	0	0	0	0
	11	1	1	1	1
	10	1	1	1	1

Minimierte DNF:

$S_3 \vee S_2$

$S_3 \vee (S_1 \wedge \overline{S_2})$

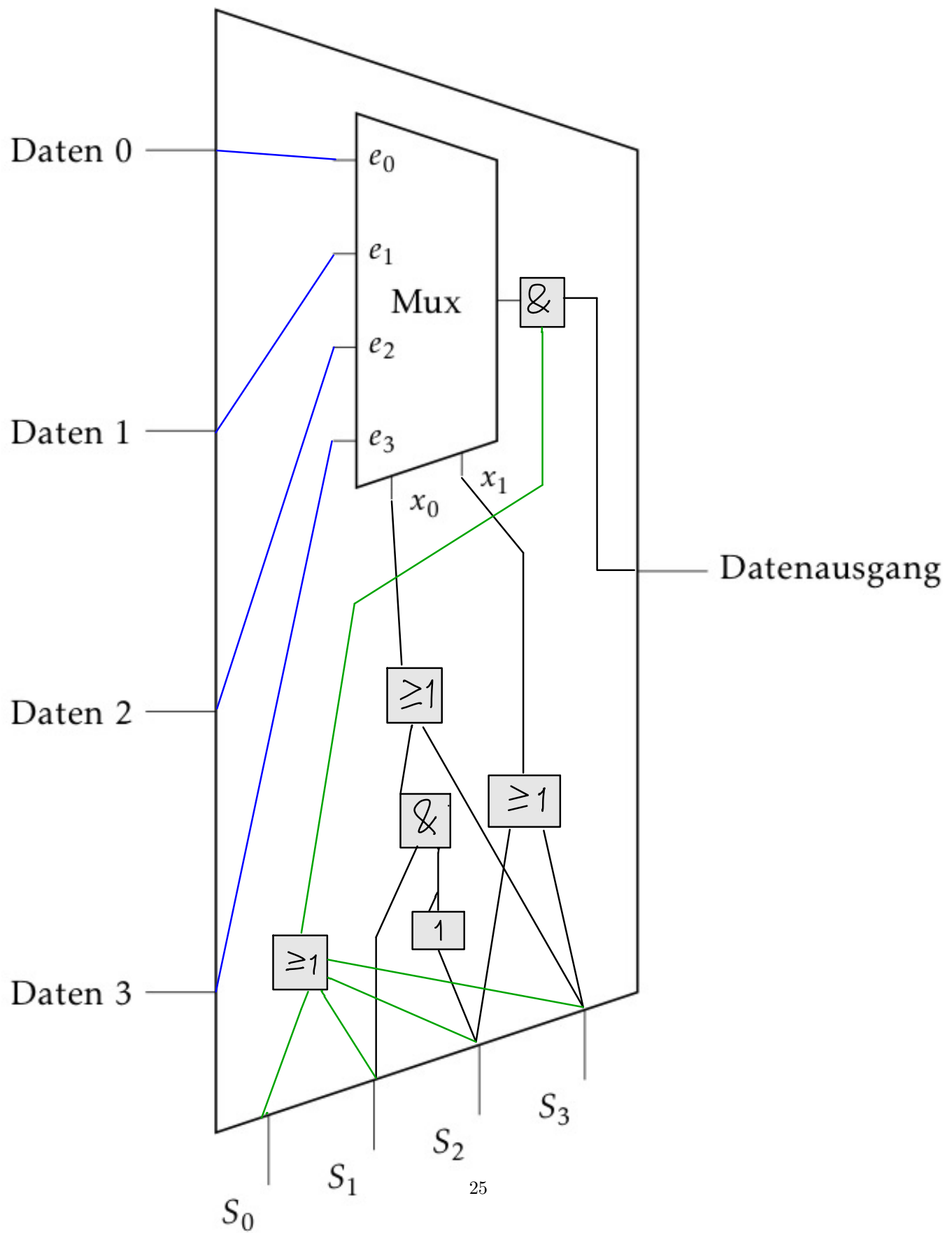
1) $X_1 = (S_3 \vee S_2)$

2) $X_0 = S_3 \vee (S_1 \wedge \overline{S_2})$

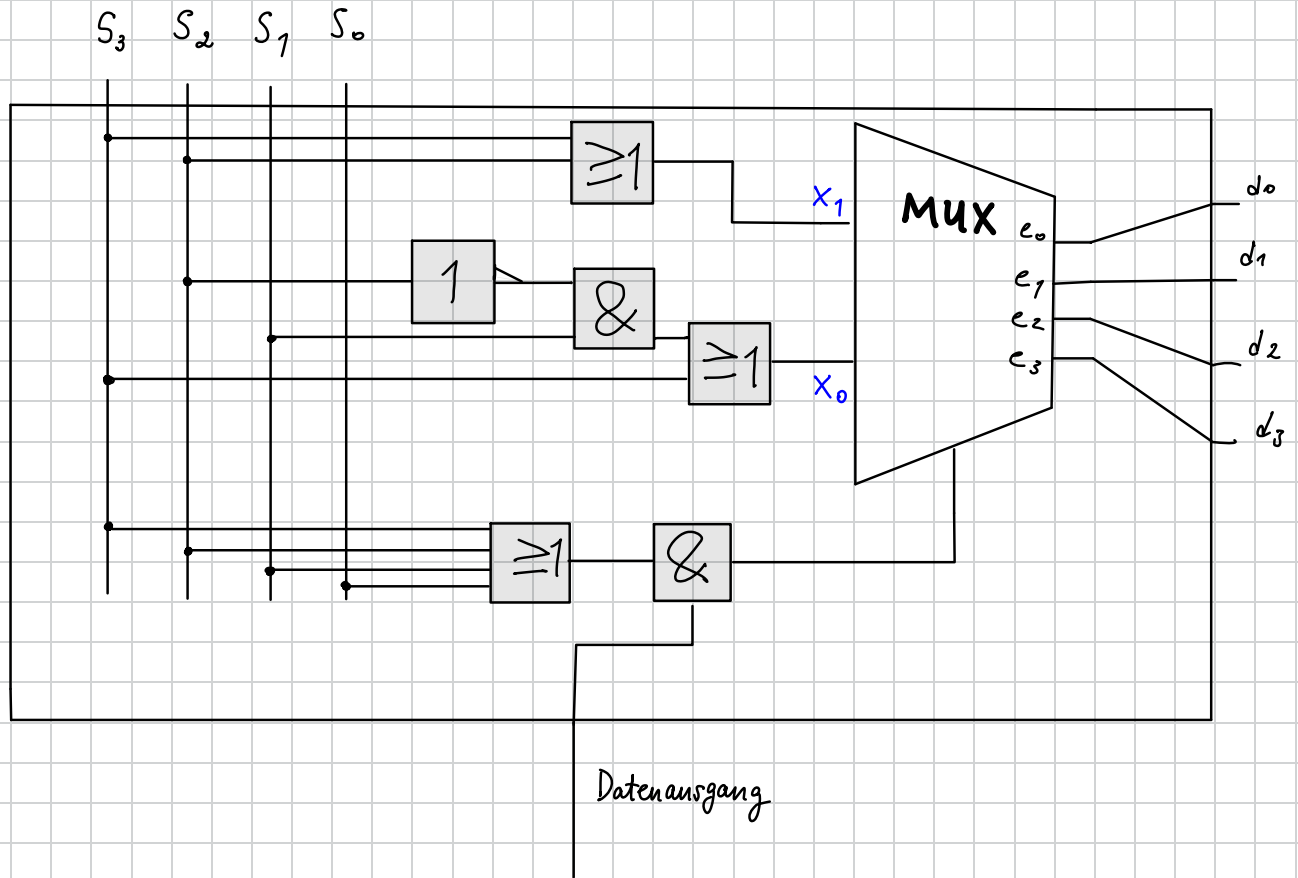
3) $S_3 \vee S_2 \vee S_1 \vee S_0 = 0$

$0 \vee 0 \vee 0 \vee 0 = 0$

$0 \wedge \text{egal was von MUX} = 0$



PMUX



2. Wozu ein Prioritätsmultiplexer?

Ein Prioritäts-Multiplexer kann nützlich sein, wenn mehrere Eingabequellen um eine begrenzte Resource konkurrieren. Beispiel: ein Nutzungsszenario mit mehreren Browser-Tabs, einem offenen E-Mail Client, einem Download, einem Video-Call und einer ausgehenden ping-Anfrage. Wenn unsere Netzwerkkarte kann nicht alle ein- und ausgehenden Pakete gleichzeitig oder schnell genug verarbeiten kann, müssen Entscheidungen getroffen werden, welches Paket zuerst verarbeitet wird. Manche Verbindungen sind empfindlicher gegenüber Verzögerungen als andere: ein Video-Call leidet schon unter Verzögerungen im Millisekunden-Bereich; eine E-Mail kann in den meisten Fällen 2 Sekunden später ankommen ohne dass es auffällt; eine ping-Anfrage ergibt nur Sinn, wenn sie so schnell wie möglich weitergeleitet wird; ein Download funktioniert auch mit großer Latenz, solange die Verbindung nicht völlig unterbrochen wird.

Im Sinne der sogenannten Dienstgüte klassifiziert das Betriebssystem und/oder die Netzwerkhardware den Datenverkehr – z.B. auf Ebene des Network-Schedulers im Betriebssystem.

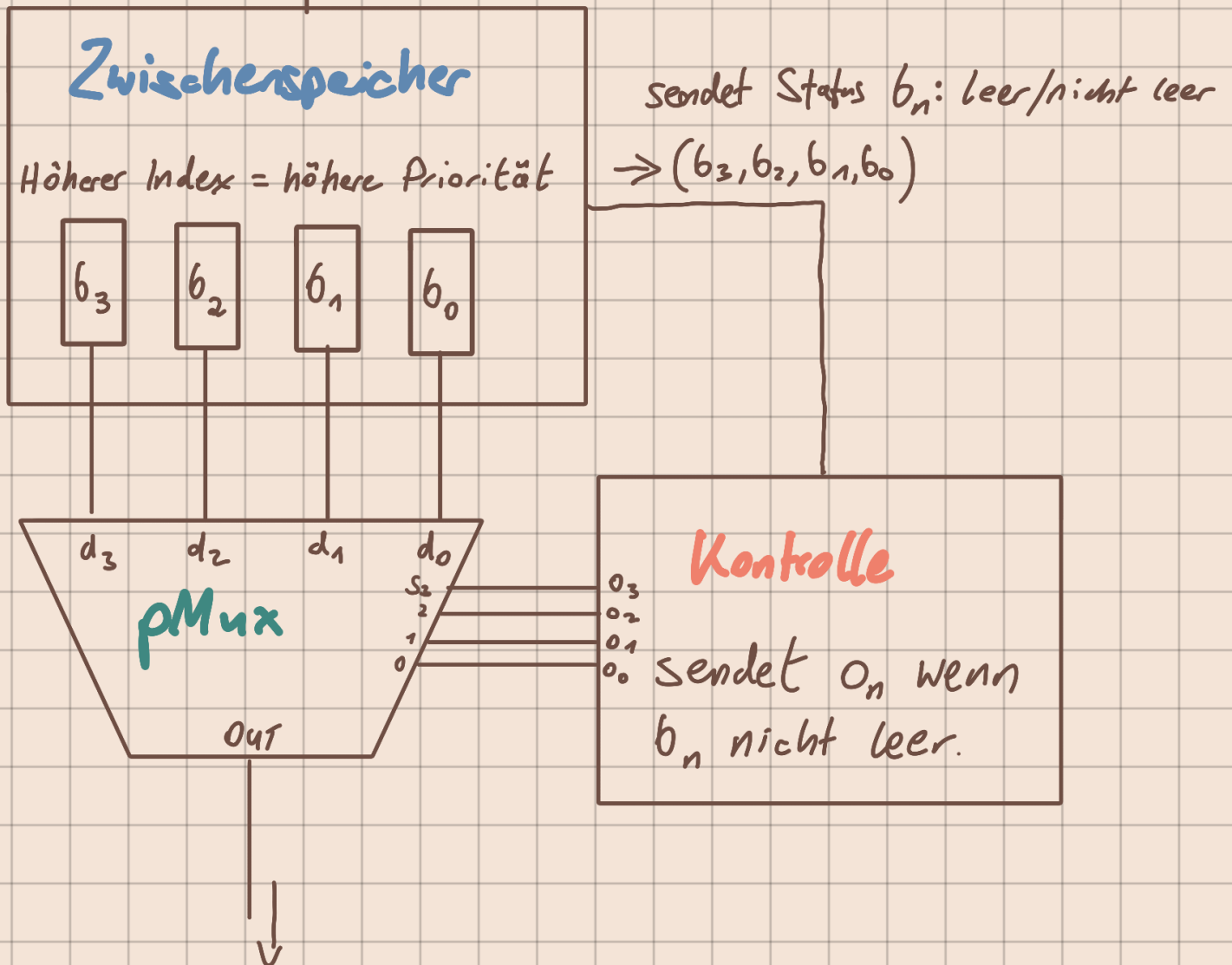
Stellen wir uns vor, was in unserem Beispiel bei Überlastung passieren könnte, sehen wir, dass ein System wie ein Prioritätsmultiplexer von Nutzen sein könnte. Bei keiner Überlastung wird jedes Paket sofort an die CPU weitergeleitet. Bei Überlastung bildet sich eine Schlange, und das System muss entscheiden, welches Paket zuerst weitergeleitet wird – z.B. unser HTTP-Request oder doch eher das UDP-Paket des Video-Calls?

Sehr vereinfacht könnten wir uns das als eine Reihe von Zwischenspeichern vorstellen, die mit den Dateneingängen des pMUX verbunden sind (**siehe S. 28: Diagramm "Paketverarbeitung mit PMUX"**). Eingehende Pakete werden nach Typ klassifiziert und dem entsprechenden Zwischenspeicher zugeordnet. UDP-Pakete eines Video-Calls landen in einem höher priorisierten Speicher als TCP-Pakete eines HTTP-Requests. Die Speicher sind mit einem Kontroller verbunden, der an die Steuereingänge des pMUX weiterleitet: AN wenn der entsprechende Speicher nicht leer ist, und ansonsten AUS.

Resultat: der Zwischenspeicher mit der höchsten Priorität wird bevorzugt geleert. Wenn dieser Speicher leer ist, gibt der Kontroller auf diesem Kanal eine 0 aus und der pMUX leitet nun die Eingabe mit der nächsthöheren Priorität weiter.

Paketverarbeitung mit pMUX

IN: Paket, wird vom System
↓
einem Zwischenspeicher zugewiesen.



OUT: Paket mit
höchster Prio.